Ex. No: 14

Title: Unfolded RNN Architecture Visualization

Packages: TensorFlow, Keras, NetworkX, Matplotlib

AIM:

To build a simple RNN model using Keras and to visualize its unfolded architecture over time using NetworkX.

ALGORITHM:

1. Start the program and import NetworkX, Matplotlib and Keras.
2. Build a simple RNN model with 4 time steps and print its summary.
3. Create a directed graph using NetworkX.
4. Add the nodes for input (x), hidden state (h) and output (y) for each time step.
5. Add the edges: x to h, h to y and h(t-1) to h(t).
6. Set the position of each node and draw the graph.
7. Display the output and stop.

PROGRAM:

```
import networkx as nx
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers

# A simple RNN model with 4 time steps
model = keras.Sequential([
    layers.Input(shape=(4, 1)),
    layers.SimpleRNN(3),
    layers.Dense(1)
])
model.summary()

# Unfolded RNN graph using NetworkX
G = nx.DiGraph()
pos = {}
for t in range(1, 5):
    G.add_edge("x" + str(t), "h" + str(t))      # input -> hidden
    G.add_edge("h" + str(t), "y" + str(t))      # hidden -> output
    if t > 1:
        G.add_edge("h" + str(t - 1), "h" + str(t))   # hidden -> next hidden
    pos["x" + str(t)] = (t, 0)
    pos["h" + str(t)] = (t, 1)
    pos["y" + str(t)] = (t, 2)

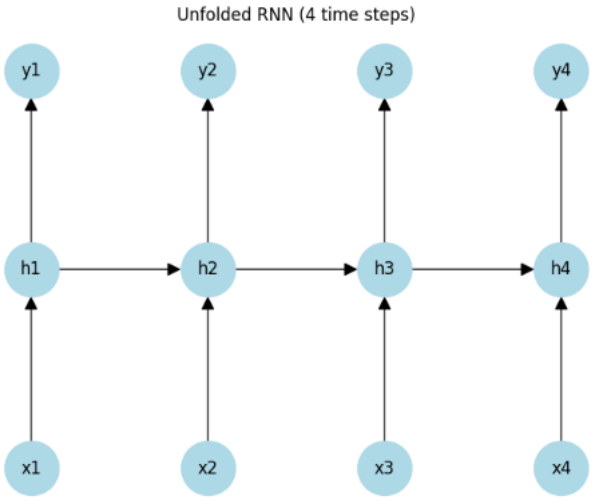
nx.draw(G, pos, with_labels=True, node_size=1500, node_color="lightblue", arrowsize=20)
plt.title("Unfolded RNN (4 time steps)")
plt.show()
```

OUTPUT:

Model: "sequential_11"

Layer (type)	Output Shape	Param #
simple_rnn_1 (SimpleRNN)	(None, 3)	15
dense_26 (Dense)	(None, 1)	4

Total params: 19 (76.00 B)
Trainable params: 19 (76.00 B)
Non-trainable params: 0 (0.00 B)



RESULT:

The unfolded RNN architecture was visualized successfully using NetworkX, showing how the hidden state is passed from one time step to the next.